

Chapter 17: java.sql package

- MySQL
- JDBC Driver Types
- Steps for Database Connectivity
- Statement, PreparedStatement and CallableStatement
- ResultSet
- Transactions
- Batch Processing
- Streaming

Downloading MySQL Server

<https://dev.mysql.com/downloads/installer/>

MySQL Server

1. Execute the Setup.
2. Select Custom Installation, click on Next.
3. Within Check Requirements, click on "Execute" and install all the requirements and then click on Finish.

NOTE: MySQL Server, MySQL WorkBench and MySQL JConnector are the only required tools, remaining can be ignored.

To Verify

- Start | Run | MySQL 8.0 Command Line Client
- Specify the password and press enter key.

MySQL WorkBench

- It is tool/utility used to work on MySQL Server
- To Open MySQL WorkBench
Start | Run | MySQL WorkBench

To Create a New Database/Schema

1. Within Navigator | Schemas | Create Schema
2. Specify the Schema Name
3. Click on Apply

To perform any operation on Schema, it needs to be activated or selected

Within Navigator | Schemas | Select the Schema | Set As Default Schema

To Backup the Schema

1. Within Navigator | Data Export
2. Select the Schema and Objects to be exported
3. Select Export to Self Contained File
4. Specify the name and location of the SQL file by clicking on the ...
5. Select Start Export

To Import/Restore the Schema

1. Within Navigator | Data Import/Restore
2. Select Import from Self Contained File

3. Specify the name and location of the SQL file by clicking on the ...
4. Select Start Import

To Create a New Table

1. Within Navigator | Schemas | Select the Schema | Select Tables | Create Table
2. Specify Table Name and column details and click on Apply

To Alter/Delete/Truncate Table

Within Navigator | Schemas | Select the Schema | Select Tables | Alter/Delete/Truncate Table

Sample Table: categories

```
categoryid    int
categoryname  varchar(45)
categorydetails varchar(100)
```

SQL ANCD
mysql
oracle data
Deby
DB2
MS SQL SERVER
Sybase

Insert: To insert row in a table

1. Inserting values of all columns

Insert into categories values(1,'Chocolates','Sweet Chocolates');

2. Inserting values of selected columns

Insert into categories(categoryname, categorydetails) values('Ice-Cream','Frozen Icecream');

id यदि Auto number होगा तो 1 के बाद दो आया होगा।

Delete: To Delete rows of a table

1. Deleting all rows

Delete from categories;

↑ keyword

2. Deleting Selected rows

Delete from categories where categoryid=1;

Table की name
होती है फिलर

जिसकी id=1 है उसको delete करेगा।

Update: To Update rows of a Table

1. Updating all rows

update categories set categorydetails=";

दो बार single quote है! सारा खाना ही गलत करेगा कि where नहीं लगाये।

2. Updating a selected row

update categories set categoryname='Wafers', categorydetails='Salted Wafers' where categoryid=1;

Select: To fetch rows of a table

1. All rows, All Columns

Select * from categories;

→ सारा column.

2. All rows, selected Columns

Select categoryname from categories;

3. Selected rows, all columns

Select * from categories where categoryid=1;

Java database connective
open

4. Selected row, Selected columns

Select categoryname from categories where categoryid=1;

What is JDBC Driver?

JDBC drivers implement the defined interfaces in the JDBC API, for interacting with your database server. For example, using JDBC drivers enable you to open database connections and to interact with it by sending SQL or database commands then receiving results with Java.

The Java.sql package that ships with JDK, contains various classes with their behaviours defined and their actual implementations are done in third-party drivers. Third party vendors implement the java.sql.Driver interface in their database driver.

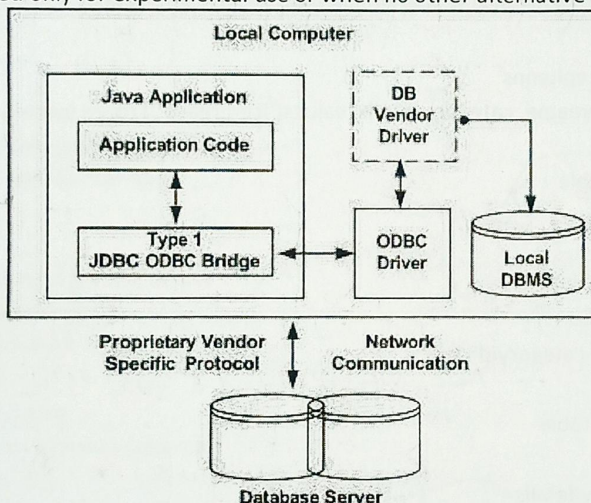
JDBC Drivers Types

JDBC driver implementations vary because of the wide variety of operating systems and hardware platforms in which Java operates. Sun has divided the implementation types into four categories, Types 1, 2, 3, and 4, which is explained below -

Type 1: JDBC-ODBC Bridge Driver

In a Type 1 driver, a JDBC bridge is used to access ODBC drivers installed on each client machine. Using ODBC, requires configuring on your system a Data Source Name (DSN) that represents the target database.

When Java first came out, this was a useful driver because most databases only supported ODBC access but now this type of driver is recommended only for experimental use or when no other alternative is available.



The JDBC-ODBC Bridge that comes with JDK 1.2 is a good example of this kind of driver.

Type 2: JDBC-Native API

In a Type 2 driver, JDBC API calls are converted into native C/C++ API calls, which are unique to the database. These drivers are typically provided by the database vendors and used in the same manner as the JDBC-ODBC Bridge. The vendor-specific driver must be installed on each client machine.

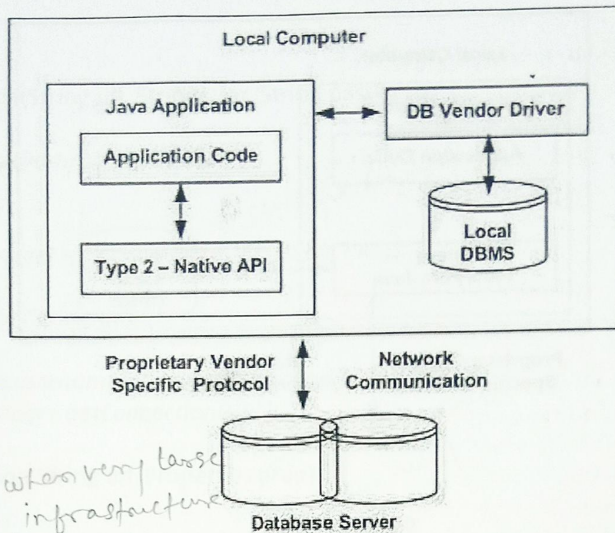
If we change the Database, we have to change the native API, as it is specific to a database and they are mostly obsolete now, but you may realize some speed increase with a Type 2 driver, because it eliminates ODBC's overhead. The Oracle Call Interface (OCI) driver is an example of a Type 2 driver.

ODBC → Any Language
→ Any database
→ only for windows m/c

JDBC → Any Database
→ Any Platform

Java.SQL.Driver ← interface for making own Driver.

Driver on which connection establish krni krna hoga
 (31/11/21)



Java is Database independent.

Driver is Database dependent.

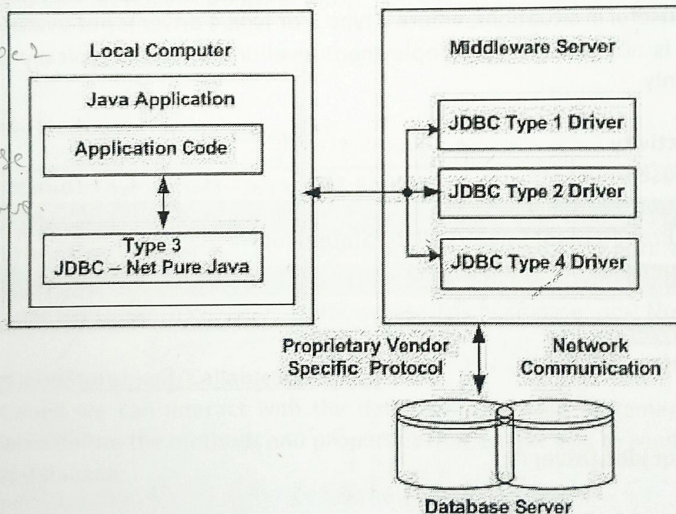
Database independent

use when very large infrastructure

Type 3: JDBC-Net pure Java

In a Type 3 driver, a three-tier approach is used to access databases. The JDBC clients use standard network sockets to communicate with a middleware application server. The socket information is then translated by the middleware application server into the call format required by the DBMS, and forwarded to the database server. This kind of driver is extremely flexible, since it requires no code installed on the client and a single driver can actually provide access to multiple databases.

Net pure Java.
 Bundle of Type 1, Type 2 and Type 4
 → multiple Database
 → middleWare Server.



You can think of the application server as a JDBC "proxy," meaning that it makes calls for the client application. As a result, you need some knowledge of the application server's configuration in order to effectively use this driver type. Your application server might use a Type 1, 2, or 4 driver to communicate with the database, understanding the nuances will prove helpful.

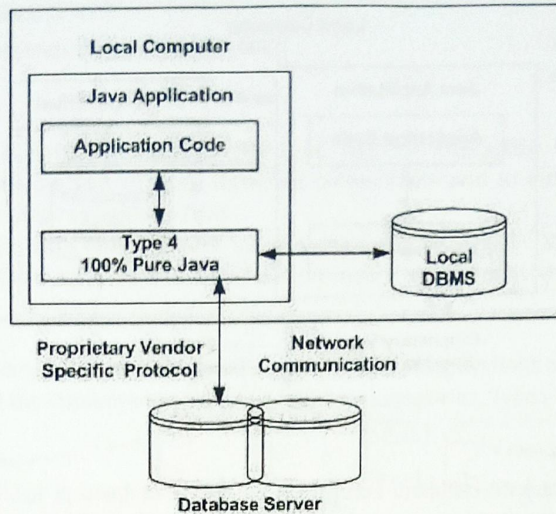
Type 4: 100% Pure Java → for our project this is good - Database dependent

In a Type 4 driver, a pure Java-based driver communicates directly with the vendor's database through socket connection. This is the highest performance driver available for the database and is usually provided by the vendor itself.

This kind of driver is extremely flexible, you don't need to install special software on the client or server. Further, these drivers can be downloaded dynamically.

Latest / fastest
 No Installation

Java files in form E.



MySQL's Connector/J driver is a Type 4 driver. Because of the proprietary nature of their network protocols, database vendors usually supply type 4 drivers.

Which Driver should be Used?

- If you are accessing one type of database, such as Oracle, Sybase, or IBM, the preferred driver type is 4.
- If your Java application is accessing multiple types of databases at the same time, type 3 is the preferred driver.
- Type 2 drivers are useful in situations, where a type 3 or type 4 driver is not available yet for your database.
- The type 1 driver is not considered a deployment-level driver, and is typically used for development and testing purposes only.

Steps for Database Connectivity

1. Register the Database Driver
2. Create the Connection
3. Create Statement/PreparedStatement/CallableStatement
4. execute/executeQuery/executeUpdate

I. Register the Database Driver

Approach I - Class.forName()

```

try {
    Class.forName("com.mysql.jdbc.Driver");
}
catch(ClassNotFoundException ex) {
    System.out.println("Error: unable to load driver class!");
}
    
```

Approach II - DriverManager.registerDriver()

```

try {
    Driver myDriver = new com.mysql.jdbc.Driver();
    DriverManager.registerDriver(myDriver);
}
catch(ClassNotFoundException ex) {
    System.out.println("Error: unable to load driver class!");
}
    
```


Connection → interface ?

II. Create the Connection

Option 1: `getConnection(String url, String user, String password)`

```
String URL = "jdbc:oracle:thin:@amrood:1521:EMP";
String USER = "username";
String PASS = "password"
Connection conn = DriverManager.getConnection(URL, USER, PASS);
```

→ static function

Option 2: `getConnection(String url)`

```
String URL = "jdbc:oracle:thin:username/password@amrood:1521:EMP";
Connection conn = DriverManager.getConnection(URL);
```

→ static function

Option 3: `getConnection(String url, Properties prop)`

```
import java.util.*;

String URL = "jdbc:oracle:thin:@amrood:1521:EMP";
Properties info = new Properties( );
info.put( "user", "username" );
info.put( "password", "password" );
Connection conn = DriverManager.getConnection(URL, info);
```

→ java.util

Closing JDBC Connections

`conn.close()`

RDBMS	JDBC Driver Name	URL Format
MySQL	com.mysql.jdbc.Driver	jdbc:mysql://hostname/ databaseName
Oracle	oracle.jdbc.driver.OracleDriver	jdbc:oracle:thin:@hostname:port Number:databaseName
DB2	com.ibm.db2.jdbc.net.DB2Driver	jdbc:db2:hostname:port Number/databaseName
Sybase	com.sybase.jdbc.SybDriver	jdbc:sybase:Tds:hostname: port Number/databaseName

III. Create Statement/PreparedStatement/CallableStatement

Once a connection is obtained we can interact with the database. The JDBC Statement, CallableStatement, and PreparedStatement interfaces define the methods and properties that enable you to send SQL or PL/SQL commands and receive data from your database.

They also define methods that help bridge data type differences between Java and SQL data types used in a database.

Interfaces	Recommended Use
Statement	Use the for general-purpose access to your database. Useful when you are using static SQL statements at runtime. The Statement interface cannot accept parameters.
PreparedStatement	Use the when you plan to use the SQL statements many times. The PreparedStatement interface accepts input parameters at runtime.
CallableStatement	Use the when you want to access the database stored procedures. The CallableStatement interface can also accept runtime input parameters.

variable, value → Statement
variable, value → PreparedStatement

Statement

```
try (Statement stmt = conn.createStatement()) {  
    String SQL = "Update Employees SET age = 35 WHERE id = 101";  
    int x = stmt.executeUpdate(SQL);  
}  
catch (SQLException e) {  
    ...  
}
```

Once you've created a Statement object, you can then use it to execute an SQL statement with one of its three execute methods.

boolean execute (String SQL): Returns a boolean value of true if a ResultSet object can be retrieved; otherwise, it returns false. Use this method to execute SQL DDL statements or when you need to use truly dynamic SQL.

int executeUpdate (String SQL): Returns the number of rows affected by the execution of the SQL statement. Use this method to execute SQL statements for which you expect to get a number of rows affected - for example, an INSERT, UPDATE, or DELETE statement.

ResultSet executeQuery (String SQL): Returns a ResultSet object. Use this method when you expect to get a result set, as you would with a SELECT statement.

JM1-SQLSyntax: Pre-Requisites

1. Restore the database - JDBC Examples – Eclipse\Database.sql
2. Open within Eclipse Project
3. To set CLASSPATH with Eclipse
 - Project | Properties | Java Build Path | Libraries | External Jar Files
 - Select the MySQLConnector.jar from C:\Program Files (x86)\MySQL\Connector J 8.0
4. Within the Program change the password

Example: JM1-SQLSyntax\FirstExample.java

PreparedStatement

```
String SQL = "Update Employees SET age = ? WHERE id = ?";  
try (PreparedStatement pstmt = conn.prepareStatement(SQL)) {  
    pstmt.setInt(1, 35);  
    pstmt.setInt(2, 101);  
    int x = pstmt.executeUpdate();  
}  
catch (SQLException e) {  
    ...  
}
```

Example: JM1-SQLSyntax\PSEExample.java

CallableStatement

```
String SQL = "{call getEmpName (?, ?)}";  
try (CallableStatement cstmt = conn.prepareCall (SQL)) {
```


इस तरह SQL prepared statement में query को pass कर दिया है जो
 दोबारा execute/update() में मत pass करना

```
...
}
catch (SQLException e) {
    ...
}
```

Example: JM1-SQLSyntax \CSEExample.java

Sample Oracle Procedure

```
CREATE OR REPLACE PROCEDURE getEmpName
(EMP_ID IN NUMBER, EMP_FIRST OUT VARCHAR) AS
BEGIN
    SELECT first INTO EMP_FIRST
    FROM Employees
    WHERE ID = EMP_ID;
END;
```

Sample MySQL Procedure

```
DELIMITER $$
DROP PROCEDURE IF EXISTS `emp`.`getEmpName` $$
CREATE PROCEDURE `emp`.`getEmpName`
(IN EMP_ID INT, OUT EMP_FIRST VARCHAR(255))
BEGIN
    SELECT first INTO EMP_FIRST
    FROM Employees
    WHERE ID = EMP_ID;
END $$
DELIMITER ;
```

ResultSet

The SQL statements that read data from a database query, return the data in a result set. The SELECT statement is the standard way to select rows from a database and view them in a result set. The java.sql.ResultSet interface represents the result set of a database query.

A ResultSet object maintains a cursor that points to the current row in the result set. The term "result set" refers to the row and column data contained in a ResultSet object.

The methods of the ResultSet interface can be broken down into three categories –

Navigational methods: Used to move the cursor around.

Get methods: Used to view the data in the columns of the current row being pointed by the cursor.

Update methods: Used to update the data in the columns of the current row. The updates can then be updated in the underlying database as well.

JDBC provides the following connection methods to create statements with desired ResultSet –

```
createStatement(int RSType, int RSConcurrency);
prepareStatement(String SQL, int RSType, int RSConcurrency);
prepareCall(String sql, int RSType, int RSConcurrency);
```


Type of ResultSet

The possible RSType are given below. If you do not specify any ResultSet type, you will automatically get one that is TYPE_FORWARD_ONLY.

Type	Description
ResultSet.TYPE_FORWARD_ONLY	The cursor can only move forward in the result set.
ResultSet.TYPE_SCROLL_INSENSITIVE	The cursor can scroll forward and backward, and the result set is not sensitive to changes made by others to the database that occur after the result set was created.
ResultSet.TYPE_SCROLL_SENSITIVE	The cursor can scroll forward and backward, and the result set is sensitive to changes made by others to the database that occur after the result set was created.

Concurrency of ResultSet

The possible RSConcurrency are given below. If you do not specify any Concurrency type, you will automatically get one that is CONCUR_READ_ONLY.

Concurrency	Description
ResultSet.CONCUR_READ_ONLY	Creates a read-only result set. This is the default
ResultSet.CONCUR_UPDATABLE	Creates an updateable result set.

Navigating a Result Set

public void beforeFirst() throws SQLException
 public void afterLast() throws SQLException
 public boolean first() throws SQLException
 public boolean last() throws SQLException
 public boolean absolute(int row) throws SQLException
 public boolean relative(int row) throws SQLException
 public boolean previous() throws SQLException
 public boolean next() throws SQLException
 public int getRow() throws SQLException
 public void moveToInsertRow() throws SQLException
 public void moveToCurrentRow() throws SQLException

Viewing a Result Set

public XXX getXXX(String columnName) throws SQLException
 public XXX getXXX(int columnIndex) throws SQLException

Example: JM1-SQLSyntax\RSExample1.java

Updating a Result Set

public void updateXXX(int columnIndex, XXX value) throws SQLException
 public void updateXXX(String columnName, XXX value) throws SQLException
 public void updateRow()
 public void deleteRow()
 public void refreshRow()
 public void cancelRowUpdates()
 public void insertRow()

Example: JM1-SQLSyntax\RSExample2.java

JDBC - Transactions

If your JDBC Connection is in auto-commit mode, which it is by default, then every SQL statement is committed to the database upon its completion.

That may be fine for simple applications, but there are three reasons why you may want to turn off the auto-commit and manage your own transactions –

- To increase performance.
- To maintain the integrity of business processes.
- To use distributed transactions.

Transactions enable you to control if, and when, changes are applied to the database. It treats a single SQL statement or a group of SQL statements as one logical unit, and if any statement fails, the whole transaction fails.

To enable manual- transaction support instead of the auto-commit mode that the JDBC driver uses by default, use the Connection object's `setAutoCommit()` method. If you pass a boolean `false` to `setAutoCommit()`, you turn off auto-commit. You can pass a boolean `true` to turn it back on again.

For example, if you have a Connection object named `conn`, code the following to turn off auto-commit –

```
conn.setAutoCommit(false);
```

Commit & Rollback

Once you are done with your changes and you want to commit the changes then call `commit()` method on connection object as follows –

```
conn.commit();
```

Otherwise, to roll back updates to the database made using the Connection named `conn`, use the following code –

```
conn.rollback();
```

The following example illustrates the use of a commit and rollback object –

```
try{
    //Assume a valid connection object conn
    conn.setAutoCommit(false);
    Statement stmt = conn.createStatement();
    String SQL = "INSERT INTO Employees VALUES (106, 20, 'Rita', 'Tez')";
    stmt.executeUpdate(SQL);
    //Submit a malformed SQL statement that breaks
    String SQL = "INSERTED IN Employees VALUES (107, 22, 'Sita', 'Singh')";
    stmt.executeUpdate(SQL);
    // If there is no error.
    conn.commit();
}catch(SQLException se){
    // If there is any error.
    conn.rollback();
}
```

Example: JM1-SQLSyntax \TSEExample1.java

Using Savepoints

The new JDBC 3.0 Savepoint interface gives you the additional transactional control. Most modern DBMS, support savepoints within their environments such as Oracle's PL/SQL.

When you set a savepoint you define a logical rollback point within a transaction. If an error occurs past a savepoint, you can use the rollback method to undo either all the changes or only the changes made after the savepoint.

The Connection object has two new methods that help you manage savepoints –

setSavepoint(String savepointName): Defines a new savepoint. It also returns a Savepoint object.

releaseSavepoint(Savepoint savepointName): Deletes a savepoint. Notice that it requires a Savepoint object as a parameter. This object is usually a savepoint generated by the setSavepoint() method.

There is one rollback (**String savepointName**) method, which rolls back work to the specified savepoint.

The following example illustrates the use of a Savepoint object –

```
try{
    //Assume a valid connection object conn
    conn.setAutoCommit(false);
    Statement stmt = conn.createStatement();

    //set a Savepoint
    Savepoint savepoint1 = conn.setSavepoint("Savepoint1");
    String SQL = "INSERT INTO Employees VALUES (106, 20, 'Rita', 'Tez')";
    stmt.executeUpdate(SQL);
    //Submit a malformed SQL statement that breaks
    String SQL = "INSERTED IN Employees VALUES (107, 22, 'Sita', 'Tez')";
    stmt.executeUpdate(SQL);
    // If there is no error, commit the changes.
    conn.commit();
}catch(SQLException se){
    // If there is any error.
    conn.rollback(savepoint1);
}
```

Example: JM1-SQLSyntax\TSEExample2.java

JDBC - Batch Processing

Batch Processing allows you to group related SQL statements into a batch and submit them with one call to the database.

When you send several SQL statements to the database at once, you reduce the amount of communication overhead, thereby improving performance.

- JDBC drivers are not required to support this feature. You should use the **DatabaseMetaData.supportsBatchUpdates()** method to determine if the target database supports batch update processing. The method returns true if your JDBC driver supports this feature.
- The **addBatch()** method of Statement, PreparedStatement, and CallableStatement is used to add individual statements to the batch. The **executeBatch()** is used to start the execution of all the statements grouped together.
- The **executeBatch()** returns an array of integers, and each element of the array represents the update count for the respective update statement.
- Just as you can add statements to a batch for processing, you can remove them with the **clearBatch()** method. This method removes all the statements you added with the addBatch() method. However, you cannot selectively choose which statement to remove.

Batching with Statement Object

Here is a typical sequence of steps to use Batch Processing with Statement Object –

1. Create a Statement object using either createStatement() methods.
2. Set auto-commit to false using setAutoCommit().
3. Add as many as SQL statements you like into batch using addBatch() method on created statement object.
4. Execute all the SQL statements using executeBatch() method on created statement object.
5. Finally, commit all the changes using commit() method.

Example

The following code snippet provides an example of a batch update using Statement object –

```
// Create statement object
Statement stmt = conn.createStatement();

// Set auto-commit to false
conn.setAutoCommit(false);

// Create SQL statement
String SQL = "INSERT INTO Employees (id, first, last, age) VALUES(200,'Zia', 'Ali', 30)";
// Add above SQL statement in the batch.
stmt.addBatch(SQL);

// Create one more SQL statement
String SQL = "INSERT INTO Employees (id, first, last, age) VALUES(201,'Raj', 'Kumar', 35)";
// Add above SQL statement in the batch.
stmt.addBatch(SQL);

// Create one more SQL statement
String SQL = "UPDATE Employees SET age = 35 WHERE id = 100";
// Add above SQL statement in the batch.
stmt.addBatch(SQL);

// Create an int[] to hold returned values
int[] count = stmt.executeBatch();

//Explicitly commit statements to apply changes
conn.commit();
```

Example: JM1-SQLSyntax \BPExample1.java

Batching with PreparedStatement Object

Here is a typical sequence of steps to use Batch Processing with PreparedStatement Object –

1. Create SQL statements with placeholders.
2. Create PreparedStatement object using either prepareStatement() methods.
3. Set auto-commit to false using setAutoCommit().
4. Add as many as SQL statements you like into batch using addBatch() method on created statement object.
5. Execute all the SQL statements using executeBatch() method on created statement object.
6. Finally, commit all the changes using commit() method.

The following code snippet provides an example of a batch update using PreparedStatement object –


```
// Create SQL statement
String SQL = "INSERT INTO Employees (id, first, last, age) VALUES(?, ?, ?, ?)";
```

```
// Create PreparedStatement object
PreparedStatement pstmt = conn.prepareStatement(SQL);
```

```
//Set auto-commit to false
conn.setAutoCommit(false);
```

```
// Set the variables
pstmt.setInt( 1, 400 );
pstmt.setString( 2, "Pappu" );
pstmt.setString( 3, "Singh" );
pstmt.setInt( 4, 33 );
// Add it to the batch
pstmt.addBatch();
```

```
// Set the variables
pstmt.setInt( 1, 401 );
pstmt.setString( 2, "Pawan" );
pstmt.setString( 3, "Singh" );
pstmt.setInt( 4, 31 );
// Add it to the batch
pstmt.addBatch();
//add more batches
```

```
//Create an int[] to hold returned values
int[] count = stmt.executeBatch();
```

```
//Explicitly commit statements to apply changes
conn.commit();
```

Example: JM1-SQLSyntax \BPExample2.java

JDBC - Streaming ASCII and Binary Data

A PreparedStatement object has the ability to use input and output streams to supply parameter data. This enables you to place entire files into database columns that can hold large values, such as CLOB and BLOB data types.

There are following methods, which can be used to stream data –

setAsciiStream(): This method is used to supply large ASCII values.

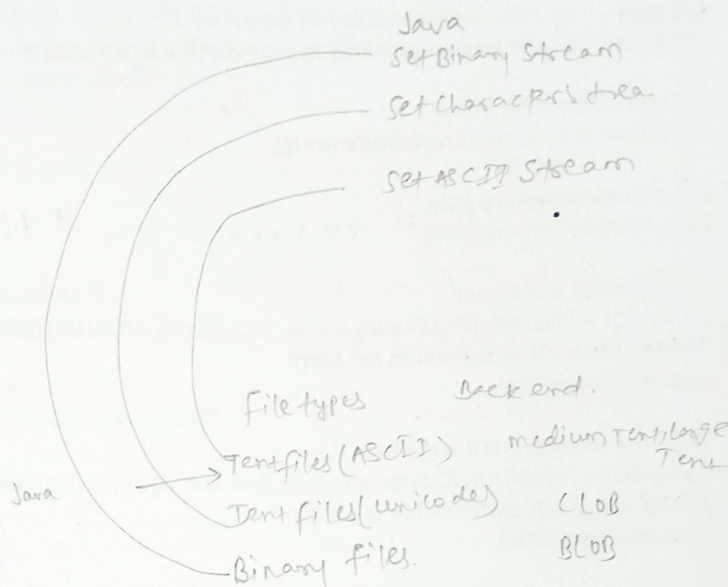
setCharacterStream(): This method is used to supply large UNICODE values.

setBinaryStream(): This method is used to supply large binary values.

The setXXXStream() method requires an extra parameter, the file size, besides the parameter placeholder. This parameter informs the driver how much data should be sent to the database using the stream.

Example: JM1-SQLSyntax \AsciiStreamExample.java

Example: JM1-SQLSyntax \BinaryStreamExample.java



SQL	JDBC/Java	setXXX	getXXX	updateXXX
VARCHAR	java.lang.String	setString	getString	updateString
CHAR	java.lang.String	setString	getString	updateString
LONGVARCHAR	java.lang.String	setString	getString	updateString
BIT	Boolean	setBoolean	getBoolean	updateBoolean
NUMERIC	java.math.BigDecimal	setBigDecimal	getBigDecimal	updateBigDecimal
TINYINT	Byte	setByte	getByte	updateByte
SMALLINT	Short	setShort	getShort	updateShort
INTEGER	Int	setInt	getInt	updateInt
BIGINT	Long	setLong	getLong	updateLong
REAL	Float	setFloat	getFloat	updateFloat
FLOAT	Float	setFloat	getFloat	updateFloat
DOUBLE	Double	setDouble	getDouble	updateDouble
VARBINARY	byte[]	setBytes	getBytes	updateBytes
BINARY	byte[]	setBytes	getBytes	updateBytes
DATE	java.sql.Date	setDate	getDate	updateDate
TIME	java.sql.Time	setTime	getTime	updateTime
TIMESTAMP	java.sql.Timestamp	setTimestamp	getTimestamp	updateTimestamp
CLOB	java.sql.Clob	setCharacterStream	setCharacterStream	setCharacterStream
BLOB	java.sql.Blob	setBinaryStream	getBinaryStream	updateBinaryStream
ARRAY	java.sql.Array	setArray	getArray	updateArray
REF	java.sql.Ref	setRef	getRef	updateRef
STRUCT	java.sql.Struct	setStruct	getStruct	updateStruct

Data Types in SQL/Back-End	Java Equivalent
timestamp (stores date as well as time)	java.sql.Timestamp
date (stores date)	java.sql.Date
time (store time)	java.sql.Time

Front-End/Date

```
java.util.Date dt = new java.util.Date();
```

Storage (java.util.Date to java.sql.Date or java.sql.Time, java.sql.Timestamp)

```
java.util.Date dt = new java.util.Date();
```

```
long num = dt.getTime();
```

```
java.sql.Timestamp dateTime = new java.sql.Timestamp(num);
```

```
java.sql.Date date = new java.sql.Date(num);
```

```
java.sql.Time time = new java.sql.Time(num);
```

Retrival (java.sql.Date or java.sql.Time, java.sql.Timestamp to java.util.Date)

```
java.util.Date dt1 = new java.util.Date(dateTime.getTime());
```

```
java.util.Date dt2 = new java.util.Date(date.getTime());
```

```
java.util.Date dt3 = new java.util.Date(time.getTime());
```

JDBC Examples - Eclipse:

1. Restore the database - JDBC Examples – Eclipse\Database.sql
2. Open within Eclipse Project
3. To set CLASSPATH with Eclipse
 - Project | Properties | Java Build Path | Libraries | External Jar Files
 - Select the MySqlConnection.jar from C:\Program Files (x86)\MySQL\Connector J 8.0
4. Change password within com.vedisoft.jdbc.utilities.Connection.properties

com.vedisoft.jdbc.utilities

1. Connection.properties
2. MyDatabaseConnection.java

Connection conn = MyDatabaseConnection.getConnection();

3. DateUtils.java

java.util.Date acceptDate()
java.util.Date convertDate(String str)

com.vedisoft.jdbc.pojos

(Plain Old Java Objects/Beans/Entities)

1. Course.java
2. Student.java

com.vedisoft.jdbc.daos

(Data Access Objects)

1. CourseDao.java

public boolean createCourse(Beans bean)
public boolean editCourse(Beans bean)
public boolean removeCourse(int id)
public Beans find(int id)
public ArrayList<Beans> findAll()

2. StudentDao.java

public boolean createStudent(Beans bean)
public boolean editStudent(Beans bean)
public boolean removeStudent(int id)
public Beans find(int id)
public ArrayList<Beans> findAll()

Assignment

Theory Questions

1. What are the different types of JDBC drivers?
2. What are the steps for database connectivity?
3. What is the difference between Statement, PreparedStatement and CallableStatement?
4. What are the benefits of transactions? How can be implement transactions in Java?
5. What are the benefits of batch? What is the difference between batch created using Statement and batch created using PreparedStatement?
6. What is Streaming? What are the functions available in Java for streaming?

Practical Questions

1. Create a database by the name **staff** in MySQL.
2. Create two tables within the **staff** database.

a. departments

deptid	integer	auto number	primary key
deptname	varchar(45)		
deptdetails	varchar(45)		

Chapter 13: Annotations (Metadata)

(not for end users)

- Annotations was introduced with jdk5 → tool/सा त्रिकालता है समते है एन
- It enables you to embed supplemental information into a source file.
- It does not change the actions of a program; it leaves the semantics of a program unchanged.
- Supplement information can be used by various tools during both development and deployment. For example source code generator
- All annotation types automatically extend the Annotation interface.
- Any type of declaration can have an annotation associated with it. For example, classes, methods, fields, parameters, and enum constants can be annotated. Even an annotation can be annotated.

बहुत सारे tool है जिसको Annotation के through दिया जाते

Annotation Basics

An annotation is created through a mechanism based on the interface.

// A simple annotation type.

```
@interface MyAnno {
```

```
String str();
```

```
int val();
```

```
}
```

← इसका है पढ़ने का काम function की तरह करता है

Retention Policy

- A retention policy determines at what point an annotation is discarded.
- Java defines three policies, which are within the `java.lang.annotation.RetentionPolicy` enumeration. They are `SOURCE`, `CLASS`, and `RUNTIME`.
- An annotation with a retention policy of `SOURCE` is retained only in the source file and is discarded during compilation.
- An annotation with a retention policy of `CLASS` is stored in the `.class` file during compilation. It is not available through the JVM during run time. (default)
- An annotation with a retention policy of `RUNTIME` is stored in the `.class` file during compilation and is available through the JVM during run time.
- A retention policy is specified for an annotation by using one of Java's built-in annotations: `@Retention`.

```
@Retention(retention-policy)
```

default class

```
@Retention(RetentionPolicy.RUNTIME)
```

```
@interface MyAnno {
```

```
String str();
```

```
int val();
```

```
}
```

Example:

```
import java.lang.annotation.*;
```

```
import java.lang.reflect.*;
```

```
@Retention(RetentionPolicy.RUNTIME)
```

```
@interface MyAnno {
```

```
String str();
```

```
int val();
```

← इन packages Retention Policy.

← निरखने के बाद ही त्रिकाल पाठ्योक्ति अनुसार एक्जम्पल

reflection on API → कौन सा class, कौन सा static, annotation है हम सको है

```

}
class AnnotationExample1 {
    @MyAnno(str = "Annotation Example", val = 100)
    public static void myMeth() {
        AnnotationExample1 ob = new AnnotationExample1();
        try {
            Class<?> c = ob.getClass();
            Method m = c.getMethod("myMeth");
            MyAnno anno = m.getAnnotation(MyAnno.class);
            System.out.println(anno.str() + " " + anno.val());
        } catch (NoSuchMethodException exc) {
            System.out.println("Method Not Found.");
        }
    }

    public static void main(String args[]) {
        myMeth();
    }
}

```

Example:

```

import java.lang.annotation.*;
import java.lang.reflect.*;

```

```

@Retention(RetentionPolicy.RUNTIME)
@interface MyAnno {
    String str();
    int val();
}

```

```

class AnnotationExample2 {
    @MyAnno(str = "Two Parameters", val = 19)
    public static void myMeth(String str, int i) {
        AnnotationExample2 ob = new AnnotationExample2();
        try {
            Class<?> c = ob.getClass();
            Method m = c.getMethod("myMeth", String.class, int.class);
            MyAnno anno = m.getAnnotation(MyAnno.class);
            System.out.println(anno.str() + " " + anno.val());
        } catch (NoSuchMethodException exc) {
            System.out.println("Method Not Found.");
        }
    }
}

```

सारा coding static function में निरवधि है।

O/P Two Parameters, 19

Page 85


```

        myMeth();
    }
}

```

Example: (Using default values)

```

import java.lang.annotation.*; → Retention or Retention Policy,
import java.lang.reflect.*;

```

```

@Retention(RetentionPolicy.RUNTIME)

```

```

@interface MyAnno {
    String str() default "Testing";
    int val() default 9000;
}

```

```

class AnnotationExample4 {

```

```

    @MyAnno()

```

```

    public static void myMeth() {

```

```

        AnnotationExample4 ob = new AnnotationExample4();

```

```

        try {

```

```

            Class<?> c = ob.getClass();

```

```

            Method m = c.getMethod("myMeth");

```

```

            MyAnno anno = m.getAnnotation(MyAnno.class);

```

```

            System.out.println(anno.str() + " " + anno.val());

```

```

        } catch (NoSuchMethodException exc) {

```

```

            System.out.println("Method Not Found.");

```

```

        }

```

```

    }

```

```

    public static void main(String args[]) {

```

```

        myMeth();
    }
}

```

Example : (Marker)

```

import java.lang.annotation.*;

```

```

import java.lang.reflect.*;

```

```

@Retention(RetentionPolicy.RUNTIME)

```

```

@interface MyMarker { } ← marker

```

```

class Marker {

```

```

    @MyMarker

```



```
public static void myMeth() {
    Marker ob = new Marker();
    try {
        Method m = ob.getClass().getMethod("myMeth");
        if(m.isAnnotationPresent(MyMarker.class))
            System.out.println("MyMarker is present.");
    } catch (NoSuchMethodException exc) {
        System.out.println("Method Not Found.");
    }
}
```

→ whether present annotation or not.

```
public static void main(String args[]) {
    myMeth();
}
```

marker interface

A marker interface is an interface that has no methods or constants inside it. It provides runtime type information about objects, so the compiler and JVM have additional information about the objects.

Example: (Single Member)

```
import java.lang.annotation.*;
import java.lang.reflect.*;

@Retention(RetentionPolicy.RUNTIME)
@interface MySingle {
    int value();
}

class Single {
```

```
@MySingle(100)
public static void myMeth() {
    Single ob = new Single();
    try {
        Method m = ob.getClass().getMethod("myMeth");
        MySingle anno = m.getAnnotation(MySingle.class);
        System.out.println(anno.value());
    } catch (NoSuchMethodException exc) {
        System.out.println("Method Not Found.");
    }
}
```

A marker interface is also called a tagging interface.

eg. cloneable, Serializable, Remote.

```
public static void main(String args[]) {
    myMeth();
}
```


The Built-In Annotations

- Java defines many built-in annotations. Most are specialized, but nine are general purpose.
- Four are imported from `java.lang.annotation`: `@Retention`, `@Documented`, `@Target`, and `@Inherited`.
- Five—`@Override`, `@Deprecated`, `@FunctionalInterface`, `@SafeVarargs`, and `@SuppressWarnings`—are included in `java.lang`.

Type Annotations

- Beginning with JDK 8, the places in which annotations can be used has been expanded.
- Earlier, annotations were originally allowed only on declarations.
- With JDK 8, annotations can also be specified in most cases in which a type is used.
- It is possible to annotate the return type of a method, the type of this within a method, a cast, array levels, an inherited class, and a throws clause.
- It is also possible to annotate generic types, including generic type parameter bounds and generic type arguments.

Repeating Annotations

JDK 8 annotation feature enables an annotation to be repeated on the same element. This is called repeating annotations.

Example:

```
import java.lang.annotation.*;
import java.lang.reflect.*;
```

```
@Retention(RetentionPolicy.RUNTIME)
@Repeatable(MyRepeatedAnnos.class)
@interface MyAnno {
    String str() default "Testing";
    int val() default 9000;
}
```

```
@Retention(RetentionPolicy.RUNTIME)
@interface MyRepeatedAnnos {
    MyAnno[] value();
}
```

```
class RepeatAnno {
    @MyAnno(str = "First annotation", val = -1)
    @MyAnno(str = "Second annotation", val = 100)
    public static void myMeth(String str, int i) {
        RepeatAnno ob = new RepeatAnno();
        try {
            Class<?> c = ob.getClass();
            Method m = c.getMethod("myMeth", String.class, int.class);
            Annotation anno = m.getAnnotation(MyRepeatedAnnos.class);
            System.out.println(anno);
        } catch (NoSuchMethodException exc) {
```



```
        System.out.println("Method Not Found.");  
    }  
}  
  
public static void main(String args[]) {  
    myMeth("test", 10);  
}  
}
```

Restrictions on Annotations

- No annotation can inherit another.
- All methods declared by an annotation must be without parameters and they must return one of the following:
 - A primitive type, such as int or double
 - An object of type String or Class
 - An enum type
 - Another annotation type
 - An array of one of the preceding types
- Annotations cannot be generic. In other words, they cannot take type parameters.
- annotation methods cannot specify a throws clause.

public static int m1(int i, float f)
 ↓
m1(int, float)
 ↳ method signature

Chapter 12: Lambda

- Introducing Lambda Expressions
 - Lambda Expression Fundamentals
 - Functional Interfaces
 - Some Lambda Expression Examples
- Block Lambda Expressions
- Generic Functional Interfaces
- Passing Lambda Expressions as Arguments
- Lambda Expressions and Exceptions
- Method References
 - Method References to static Methods
 - Method References to Instance Methods
 - Method References with Generics
 - Constructor References

anonymous
object
function
class

Lambda

Added by JDK 8, lambda expressions (and their related features) significantly enhance Java because of two primary reasons.

- They added new syntax elements.
- They streamline the way that certain common constructs are implemented.

Introducing Lambda Expressions

- Java's implementations of lambda expressions are done using two constructs.
 1. The lambda expression.
 2. The functional interface.
- A **lambda expression** is, essentially, an anonymous (that is, unnamed) method. However, this method is not executed on its own. Instead, it is used to implement a method defined by a functional interface. Thus, a lambda expression results in a form of anonymous class. Lambda expressions are also commonly referred to as **closures**.
- A **functional interface** is an interface that contains one and only one abstract method. Normally, this method specifies the intended purpose of the interface. Thus, a functional interface typically represents a single action. A functional interface is sometimes referred to as a **SAM** type, where SAM stands for **Single Abstract Method**.

Lambda Expression Fundamentals

- The lambda expression introduces a new syntax element and operator into the Java language.

<para> -> [<Single expression> or <Block of code>]

Example:

() -> 123.45

This lambda expression takes no parameters, thus the parameter list is empty. It returns the constant value 123.45. Therefore, it is similar to the following method:

```
double myMeth() { return 123.45; }
```

Example:

() -> Math.random() * 100

(n) -> (n % 2) == 0

return 0.0 to 0.99
double 512E12
20 double 512E12 double

Functional Interfaces

interface title case follow करनी है
Coding में datatype interface का reference होता है।

- A functional interface is an interface that contains one and only one abstract method.

```
interface MyNumber {  
    double getValue();  
}
```

interface

Example:

```
interface MyNumber {  
    double getValue();  
}
```

कूट नहीं ला रहे हैं और double स्टार है।

```
class LambdaDemo1 {  
    public static void main(String args[]) {  
        MyNumber myNum;  
        myNum = () -> 123.45;  
        System.out.println("A fixed value: " + myNum.getValue());  
        myNum = () -> Math.random() * 100;  
        System.out.println("A random value: " + myNum.getValue());  
        System.out.println("Another random value: " + myNum.getValue());  
        // myNum = () -> "123.03"; // Error!  
    }  
}
```

random value
अगर fn value है तो error

Example:

```
interface NumericTest {  
    boolean test(int n);  
}
```

```
public class LambdaDemo2 {  
    public static void main(String args[]) {  
        NumericTest isEven = (n) -> (n % 2) == 0;  
        if (isEven.test(10))  
            System.out.println("10 is even");  
        if (!isEven.test(9))  
            System.out.println("9 is not even");  
  
        NumericTest isNonNeg = (n) -> n >= 0;  
        if (isNonNeg.test(1))  
            System.out.println("1 is non-negative");  
        if (!isNonNeg.test(-1))  
            System.out.println("-1 is negative");  
    }  
}
```

Example:

```
interface NumericTest2 {
```



```

    boolean test(int n, int d);
}
public class LambdaDemo3 {
    public static void main(String args[]) {
        NumericTest2 isFactor = (n, d) -> (n % d) == 0;
        if (isFactor.test(10, 2))
            System.out.println("2 is a factor of 10");
        if (!isFactor.test(10, 3))
            System.out.println("3 is not a factor of 10");
    }
}

```

Block Lambda Expressions

Example:

```

interface NumericFunc {
    int func(int n);
}
class BlockLambdaDemo {
    public static void main(String args[]) {
        // This block lambda computes the factorial of an int value.
        NumericFunc factorial = (n) -> {
            int result = 1;
            for (int i = 1; i <= n; i++)
                result = i * result;
            return result;
        };
        System.out.println("The factorial of 3 is " + factorial.func(3));
        System.out.println("The factorial of 5 is " + factorial.func(5));
    }
}

```

Example:

```

interface StringFunc {
    String func(String n);
}
class BlockLambdaDemo2 {
    public static void main(String args[]) {
        // This block lambda reverses the characters in a string.
        StringFunc reverse = (str) -> {
            String result = "";
            int i;
            for (i = str.length() - 1; i >= 0; i--)
                result += str.charAt(i);
            return result;
        };
    }
}

```



```
};
System.out.println("Lambda reversed is " + reverse.func("Lambda"));
System.out.println("Expression reversed is " + reverse.func("Expression"));
}
```

Generic Functional Interfaces

Example:

```
interface SomeFunc<T> {
    T func(T t);
}

class GenericFunctionalInterfaceDemo {
    public static void main(String args[]) {
        // Use a String-based version of SomeFunc.
        SomeFunc<String> reverse = (str) -> {
            String result = "";
            int i;
            for (i = str.length() - 1; i >= 0; i--)
                result += str.charAt(i);
            return result;
        };
        System.out.println("Lambda reversed is " + reverse.func("Lambda"));
        System.out.println("Expression reversed is " + reverse.func("Expression"));
        // Now, use an Integer-based version of SomeFunc.
        SomeFunc<Integer> factorial = (n) -> {
            int result = 1;
            for (int i = 1; i <= n; i++)
                result = i * result;
            return result;
        };
        System.out.println("The factorial of 3 is " + factorial.func(3));
        System.out.println("The factorial of 5 is " + factorial.func(5));
    }
}
```

String String

*interface my interface {
public int myfunction(
int n, int n);
}
public static int ABC (my interface
mi, int n, int n);
return mi.myfunction(n);*

Passing Lambda Expressions as Arguments

- Passing a lambda expression as an argument is a common use of lambdas.
- It is a very powerful use because it gives you a way to pass executable code as an argument to a method.

Example:

```
interface StringFunc {
    String func(String n);
}

public class LambdasAsArgumentsDemo {
    static String stringOp(StringFunc sf, String s) {
```

interface reference static function

ये वाने coding को समझाओ ये वाने input को समझाओ.

```

return sf.func(s);
}
public static void main(String args[]) {
    String inStr = "Lambdas add power to Java";
    String outStr;
    System.out.println("Here is input string: " + inStr);
    outStr = stringOp((str) -> str.toUpperCase(), inStr);
    System.out.println("The string in uppercase: " + outStr);
    outStr = stringOp((str) -> {
        String result = "";
        int i;
        for (i = 0; i < str.length(); i++)
            if (str.charAt(i) != ' ')
                result += str.charAt(i);
        return result;
    }, inStr);
    System.out.println("The string with spaces removed: " + outStr);
    StringFunc reverse = (str) -> {
        String result = "";
        int i;
        for (i = str.length() - 1; i >= 0; i--)
            result += str.charAt(i);
        return result;
    };
    System.out.println("The string reversed: " + stringOp(reverse, inStr));
}

```

इसी class का है directly.

→ static function है जिसमें
इसी ऐसे call किया है।

इस value
आ रहा है,

Removing space.

coding value stored in variable
reverse variable में store किया है

Variable में STM के मेमोरी है।

Lambda Expressions and Exceptions

Example:

```

interface DoubleNumericArrayFunc {
    double func(double[] n) throws EmptyArrayException;
}

class EmptyArrayException extends Exception {
    EmptyArrayException() {
        super("Array Empty");
    }
}

class LambdaExceptionDemo {
    public static void main(String args[]) throws EmptyArrayException {
        double[] values = { 1.0, 2.0, 3.0, 4.0 };
        DoubleNumericArrayFunc average = (n) -> {
            double sum = 0;
            if (n.length == 0)
                throw new EmptyArrayException();

```

→ disadvantage of lambda as argued

Coding की function में निम्नलिखित 3 तरह की Reference पाएँगे:

function types ① Constructor

② member function

③ Static function

```
for (int i = 0; i < n.length; i++)
    sum += n[i];
return sum / n.length;
};
System.out.println("The average is " + average.func(values));
System.out.println("The average is " + average.func(new double[0]));
}
}
```

Method References

Method References to static Methods

- To create a static method reference, use this general syntax:

ClassName::methodName

```
interface StringFunc {
    String func(String n);
}

class MyStringOps {
    static String strReverse(String str) {
        String result = "";
        int i;
        for (i = str.length() - 1; i >= 0; i--)
            result += str.charAt(i);
        return result;
    }
}

class MethodRefDemo {
    static String stringOp(StringFunc sf, String s) {
        return sf.func(s);
    }

    public static void main(String args[]) {
        String inStr = "Lambdas add power to Java";
        String outStr;
        outStr = stringOp(MyStringOps::strReverse, inStr);
        System.out.println("Original string: " + inStr);
        System.out.println("String reversed: " + outStr);
    }
}
```

→ static function directly call

Method References to Instance Methods

- To pass a reference to an instance method on a specific object, use this basic syntax:

objRef::methodName

Example:

```
interface StringFunc {
```



```
String func(String n);
}
class MyStringOps {
    String strReverse(String str) {
        String result = "";
        int i;
        for (i = str.length() - 1; i >= 0; i--)
            result += str.charAt(i);
        return result;
    }
}
class MethodRefDemo2 {
    static String stringOp(StringFunc sf, String s) {
        return sf.func(s);
    }
    public static void main(String args[]) {
        String inStr = "Lambdas add power to Java";
        String outStr;
        MyStringOps strOps = new MyStringOps();
        outStr = stringOp(strOps::strReverse, inStr);
        System.out.println("Original string: " + inStr);
        System.out.println("String reversed: " + outStr);
    }
}
```

→ member function

Constructor References

- Similar to the way that you can create references to methods, you can create references to constructors.

classname::new

```
interface MyFunc {
    MyClass func(int n);
}
class MyClass {
    private int val;
    MyClass(int v) {
        val = v;
    }
    MyClass() {
        val = 0;
    }
    int getVal() {
        return val;
    }
}
```

int jaise hi aur myclass start

int val
this.val = val

← not required

```
class ConstructorRefDemo {
```


`public static void main(String args[]) {
 MyFunc myClassCons = MyClass::new;
 MyClass mc = myClassCons.func(100);
 System.out.println("val in mc is " + mc.getVal());
 }`

Assignment: Enum, Annotations and Lambda

Enum

- Create an enum by the name **Material** that contains three constants namely Iron, Steel and Wooden. Create another enum called **FurnitureType** that can be chair, table or bed. Create a class called Furniture that stores its modelNo, modelName, modelDescription, material and type. Write a menu drive program that performs the following operations:
 - Add New Furniture (creates and adds a Furniture to ArrayList)
 - Search Furniture by modelNo
 - Search Furniture by modelName
 - Search Furniture by material
 - Search Furniture by type
 - Exit
- Create an enum call **House** that contains ^{four} three constants namely Palash, Gulmohar, Kachnar and Amaltas. These houses have their own colours Palash – Orange, Gulmohar – Red, Kachnar – Purple and Amaltas – Yellow. Create another enum called **Gender** that cans two constants Male and Female. Create a class called Student that stores his enrollmentNo, Name, Gender, class, section, and house. Write constructors, behaviour, operations to accept and display, and also override toString, equals and hashCode.

Annotations

- Create an annotation called **What** that stores a string called description. This string should also have a default value. Apply this annotation to all enums, classes and methods created in question 2. Create a separate class that contains main function. This main function should fetch description of all enum, classes and methods and displays it on screen.
- Create a marker annotation called **Special** and apply it to some of the methods of class Student created in question 2. Create a separate class that contains main function. This main function should print all the special functions of student class.
- Create an annotation called Changes that stores 3 strings – change, madeBy, and madeOn. Apply this annotation to all methods created in question 2. Create a separate class that contains main function. This main function should fetch details of changes details in various methods of question 2.

Lambda

- Create a String library class that contains the following methods:
 - public static String toTitleCase(String str)
 - public static String toCamelCase(String str)
 - public static String removeSpaces(String str)
 - public static int countVowels(String str)
 - public static int countConsonants(String str)